L'Encyclopédie des Oiseaux

Rapport de Projet — TP Web

Module	Développement Web Full-Stack
Année universitaire	2024 – 2025
Technologies	React · Flask · PostgreSQL · REST API

1. Présentation du projet

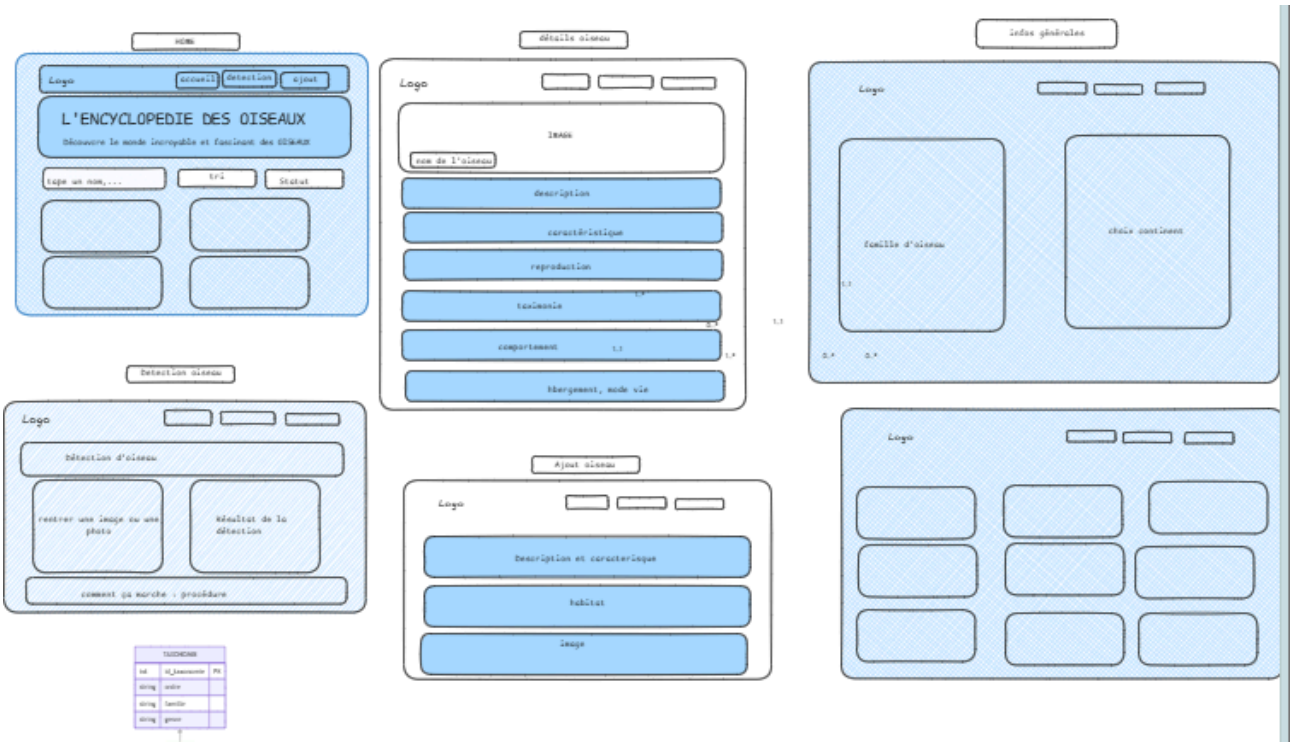
L'Encyclopédie des Oiseaux est une application web full-stack développée dans le cadre d'un TP Web. Elle permet de consulter, rechercher, comparer et identifier des espèces aviaires à travers une interface moderne et responsive.

1.1 Objectifs

- Afficher un catalogue complet d'espèces d'oiseaux avec leurs caractéristiques scientifiques.
- Permettre la recherche, le filtrage et le tri des espèces.
- Offrir une vue tableau comparative (taille, poids, envergure, longévité).
- Intégrer une détection automatique d'espèce par upload d'image (IA).
- Permettre l'ajout de nouvelles espèces et de photos via des formulaires.

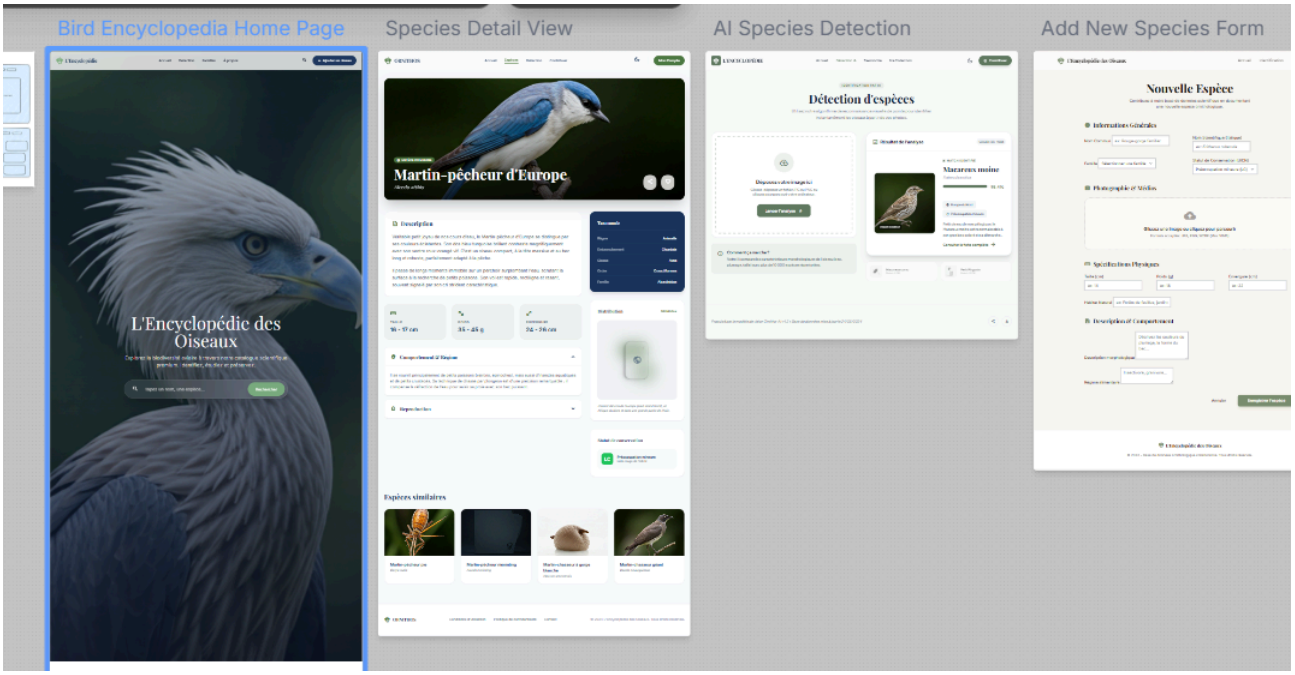
1.2 Maquettes UX et inspiration UI

La conception suit une approche mobile-first avec une charte graphique basée sur deux axes : vert forêt (#2d7a3a) et bleu nuit (#1a3a5c) pour évoquer la nature, Playfair Display pour les titres et DM Sans pour le corps.



Page	Description
Accueil	Hero plein écran photo d'oiseau, barre de recherche glassmorphique.
Catalogue	Grille 4 colonnes, filtres par statut UICN, tri par nom/taille/poids.
Détail espèce	Fiche complète : hero photo, stats, taxonomie, espèces similaires.
Tableau	Vue comparative en cartes horizontales, 4 stats par espèce.
Détection IA	Zone upload + panneau résultat avec barre de confiance.

À propos	Présentation du projet, stack technique, chiffres clés.



Voici les maquettes UI qui m'ont inspiré. Cela provient de Stitch et Figma.

2. Base de données (PostgreSQL)

2.1 Modèle Conceptuel de Données (MCD)

Le MCD identifie quatre entités principales et leurs associations :

Entité	Attributs principaux	Relations
TAXONOMIE	id_taxonomie (PK), ordre, famille, genre	1 → N espèces
ESPECE	id_espece (PK), nom_commun, nom_scientifique, taille_cm, poids_g, envergure_cm, longevite_ans, statut_conservation, habitat, description	N → 1 taxonomie1 → N images
IMAGE	id_image (PK), url, date_prise, id_espece (FK), id_auteur (FK)	N → 1 espèceN → 1 auteur
AUTEUR	id_auteur (PK), nom, email	1 → N images

2.2 Modèle Physique de Données (MPD) — PostgreSQL

PostgreSQL a été choisi pour sa robustesse, son support natif des contraintes CHECK et ses types avancés. Le schéma complet :

```
-- Taxonomie des espèces
CREATE TABLE taxonomie (
    id_taxonomie  SERIAL       PRIMARY KEY,
```

```
    ordre         VARCHAR(100) NOT NULL,
    famille       VARCHAR(100) NOT NULL,
    genre         VARCHAR(100) NOT NULL
);

-- Table principale des espèces
CREATE TABLE espece (
    id_espece            SERIAL       PRIMARY KEY,
    nom_commun           VARCHAR(150) NOT NULL,
    nom_scientifique     VARCHAR(150) NOT NULL,
    taille_cm            INTEGER,
    poids_g              INTEGER,
    envergure_cm         INTEGER,
    longevite_ans        INTEGER,
    statut_conservation  VARCHAR(2)
        CHECK (statut_conservation IN ('LC','NT','VU','EN','CR','EX')),
    habitat              TEXT,
    description          TEXT,
    id_taxonomie         INTEGER REFERENCES taxonomie(id_taxonomie)
        ON DELETE SET NULL
);

-- Auteurs / photographes
CREATE TABLE auteur (
    id_auteur  SERIAL       PRIMARY KEY,
    nom        VARCHAR(150) NOT NULL,
    email      VARCHAR(200)
);

-- Images associées aux espèces
CREATE TABLE image (
    id_image    SERIAL  PRIMARY KEY,
    url         TEXT    NOT NULL,
    date_prise  DATE,
    id_espece   INTEGER REFERENCES espece(id_espece)  ON DELETE CASCADE,
    id_auteur   INTEGER REFERENCES auteur(id_auteur)  ON DELETE SET NULL
);
```

2.3 Requêtes SQL principales

Toutes les espèces avec leur taxonomie

```
SELECT e.*, t.ordre, t.famille, t.genre
FROM espece e
LEFT JOIN taxonomie t ON e.id_taxonomie = t.id_taxonomie
ORDER BY e.nom_commun;
```

Top 20 espèces par taille décroissante

```
SELECT nom_commun, nom_scientifique, taille_cm, poids_g
FROM espece
WHERE taille_cm IS NOT NULL
```

```
ORDER BY taille_cm DESC
LIMIT 20;
```

Filtrer par statut de conservation

```
SELECT * FROM espece
WHERE statut_conservation = 'EN'
ORDER BY nom_commun;
```

Dernière image d'une espèce avec son auteur

```
SELECT i.url, i.date_prise, a.nom AS photographe
FROM image i
JOIN auteur a ON i.id_auteur = a.id_auteur
WHERE i.id_espece = $1
ORDER BY i.date_prise DESC
LIMIT 1;
```

Ajouter une espèce (requête paramétrée)

```
INSERT INTO espece
    (nom_commun, nom_scientifique, taille_cm, poids_g,
     envergure_cm, longevite_ans, statut_conservation,
     habitat, description, id_taxonomie)
VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10)
RETURNING id_espece;
```

3. Documentation API REST

L'API est construite avec Flask et psycopg2 pour la connexion à PostgreSQL. Elle est exposée sur http://localhost:5000/api et respecte les conventions REST.

3.1 Connexion Flask ↔ PostgreSQL

```
import psycopg2
from psycopg2.extras import RealDictCursor

conn = psycopg2.connect(
    host='localhost',
    database='oiseaux_db',
    user='postgres',
    password='motdepasse'
)

# Utilisation dans une route Flask
@app.route('/api/oiseaux')
def get_oiseaux():
    cur = conn.cursor(cursor_factory=RealDictCursor)
    cur.execute('SELECT * FROM espece ORDER BY nom_commun')
    data = cur.fetchall()
    return jsonify({'data': data})
```

3.2 Routes disponibles

Méthode	Route	Paramètres	Description
GET	/api/oiseaux	sort_by (optionnel)	Retourne toutes les espèces
GET	/api/oiseaux/:id	id entier	Retourne le détail d'une espèce
POST	/api/oiseaux	Corps JSON	Crée une nouvelle espèce
POST	/api/images	Corps JSON	Ajoute une image à une espèce
POST	/api/detect	multipart/form-data	Identifie une espèce par photo

3.3 Exemple de réponse — GET /api/oiseaux

```
{
  "data": [
    {
      "id_espece": 1,
      "nom_commun": "Aigle royal",
      "nom_scientifique": "Aquila chrysaetos",
      "taille_cm": 90,
      "poids_g": 4500,
      "statut_conservation": "LC",
      "habitat": "Montagnes"
    }
  ]
}
```

3.4 Codes HTTP retournés

Code	Signification
200 OK	Requête réussie, données retournées.
201 Created	Ressource créée avec succès (POST).
400 Bad Request	Corps de requête invalide ou champ manquant.
404 Not Found	Espèce ou ressource introuvable.
500 Internal Server Error	Erreur serveur (BDD, modèle IA…).

4. Difficultés rencontrées

4.1 Connexion PostgreSQL / Flask

La connexion entre Flask et PostgreSQL via psycopg2 a été la première source de problèmes :

- Erreur OperationalError : mauvais hôte, base inexistante ou mot de passe incorrect. Il a fallu créer manuellement la base avec createdb oiseaux_db et vérifier les credentials.
- Connexion non persistante : psycopg2 fermait la connexion après timeout. La solution a été d'utiliser un pool de connexions (psycopg2.pool.SimpleConnectionPool) ou de reconnecter à chaque requête.
- Encodage UTF-8 : les caractères spéciaux (accents) s'affichaient mal. Ajout de client_encoding='UTF8' dans les paramètres de connexion.
- CORS bloqué : le navigateur refusait les requêtes cross-origin. L'installation de flask-cors et l'appel à CORS(app) ont résolu le problème.

4.2 Affichage des données

- Pages vides : les composants React attendaient res.data mais l'API retournait parfois un tableau direct. Normalisation via (res.data ?? []).
- Absence d'images en base : les espèces n'avaient pas d'image_url. Une table de correspondance côté frontend (BIRD_PHOTOS) mappe le nom commun vers une URL Wikimedia Commons.
- Chargement asynchrone : sans état loading, les pages affichaient un blanc le temps de la requête. Ajout d'un useState(false) pour gérer le skeleton.
- ERR_CONNECTION_REFUSED : frontend lancé sans le backend. Nécessité de démarrer Flask ET Vite dans deux terminaux séparés.

4.3 Ajout d'image

- Champ id_espece non transmis : le paramètre query string (?espece=X) n'était pas lu. Utilisation de useSearchParams de React Router pour préremplir le champ.
- Contrainte de clé étrangère PostgreSQL : une image ne peut être insérée que si l'id_espece existe déjà en base. Un message d'erreur explicite a été ajouté côté frontend.
- Prévisualisation d'image : les URL locales ou protégées ne s'affichent pas. Seules les URL publiques sont supportées.

4.4 Détection IA (route 404)

L'endpoint de détection retournait 404 car la route dans le frontend (/api/detect) ne correspondait pas à celle définie dans Flask. La solution : tester plusieurs endpoints possibles et afficher un message d'erreur clair plutôt qu'un crash silencieux.

5. Conclusion et perspectives

5.1 Bilan

Ce projet a permis de couvrir l'ensemble de la chaîne full-stack : modélisation d'une base PostgreSQL relationnelle, API REST Flask, interface React responsive et intégration d'un module IA. Malgré les difficultés rencontrées sur la connexion, l'affichage et la gestion des images, l'application est fonctionnelle et répond aux objectifs initiaux.

5.2 Perspectives d'améliorations

Techniques

- Authentification JWT pour sécuriser les routes POST et différencier utilisateur/administrateur.
- Stockage cloud des images (Cloudinary, AWS S3) au lieu d'URL externes.
- Pagination côté serveur avec LIMIT/OFFSET PostgreSQL pour les grandes collections.
- Pool de connexions robuste avec SQLAlchemy ou psycopg2.pool.
- Tests automatisés : Pytest (backend) et Vitest (frontend).

Fonctionnelles

- Carte interactive : afficher la répartition géographique des espèces sur une carte (Leaflet.js).
- Comparaison côte à côte : permettre de sélectionner 2 à 3 espèces et de comparer leurs caractéristiques dans un tableau dédié.
- Favoris : permettre aux utilisateurs de sauvegarder leurs espèces préférées en local.
- Historique de détection : conserver un journal des identifications effectuées par l'IA avec leurs résultats.
- Mode sombre : proposer un thème sombre pour améliorer le confort de lecture.

Améliorations UX

- Squelettes de chargement (skeleton screens) pour éviter les pages blanches pendant les requêtes API.
- Notifications toast pour confirmer les actions (ajout réussi, erreur réseau) de façon non intrusive.
- Accessibilité : améliorer les attributs ARIA et la navigation clavier pour les utilisateurs en situation de handicap.

NB : Utilisation de l’IA dans ce rapport pour la mise en forme, structure du rapport et l’enrichissement. Mais les idées restent miennes.

L'Encyclopédie des Oiseaux — TP Web 2024–2025
React · Flask · PostgreSQL · REST API · Intelligence Artificielle